

G.L. Bajaj Institute of Technology and Management Greater Noida



**Department of Computer Science & Engineering
(Artificial Intelligence and Machine Learning)**

Lab File (BCAI – 551) Artificial Intelligence Lab

Submitted To:

Ms. Shruti sharma

Department of CSE(AIML)

Submitted By:

Roll No.: 2401921649002

Name: Mayank

Index

[illegible]

Experiment: 1

Aim: Write a python program to implement Breadth First Search Traversal?

Theory

BFS is a graph traversal algorithm used to explore nodes level by level.

It starts from a source node and visits all its neighbors first before moving to the next level.

BFS uses a *queue* data structure.

It works for both *trees and graphs*.

BFS helps in finding the *shortest path in an unweighted graph*.

It requires tracking visited nodes to avoid infinite loops.

BFS is widely used in *network routing, AI pathfinding*, and *social network analysis*.

Complexity

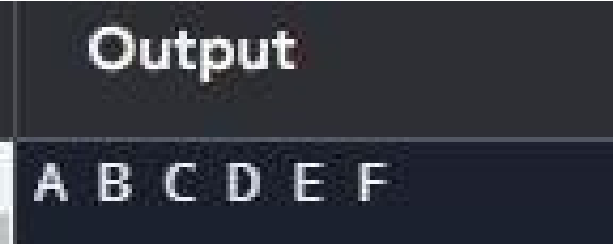
Time Complexity: $O(V + E)$

Space Complexity: $O(V)$

CODE:

```
from collections import deque
def bfs(graph, start):
    visited = set()
    queue = deque([start])
    while queue:
        node = queue.popleft()
        if node not in visited:
            print(node, end=" ")
            visited.add(node)
        for neighbour in graph[node]:
            queue.append(neighbour)

graph = {
    'A': ['B', 'C'],
    'B': ['D', 'E'],
    'C': ['F'],
    'D': [],
    'E': [],
    'F': []
}
bfs(graph, 'A')
```



Experiment: 2

Aim: Write a python program to implement Water Jug Problem?

Theory

Classic AI problem involving two jugs with different capacities.

Objective: reach a target quantity of water using only filling, emptying, and transferring.

Can be solved using **BFS** because each operation creates a new state.

States are represented as ordered pairs (x, y).

BFS explores all valid states until it reaches the target.

Used in AI to demonstrate *state-space search problem solving*.

COMPLEXITY

Time Complexity: $O(C1 * C2)$

Space Complexity: $O(C1 * C2)$ (where $C1, C2$ = jug capacities)

CODE

def water_jug():

```
visited = set()
stack = [(0, 0)] # (jug4, jug3)
while stack:
    a, b = stack.pop()
```

```
    if (a, b) in visited: continue
    visited.add((a, b)) print(a, b)
```

```
    if a == 2: # Goal: Get 2 liters break
```

```
    states = set()
    states.add((4, b))          # Fill jug 4
    states.add((a, 3))          # Fill jug 3
    states.add((0, b))          # Empty jug 4
    states.add((a, 0))          # Empty jug 3
```

```
    # Pour 4 -> 3
```

```
    pour = min(a, 3 - b) states.add((a - pour, b + pour))
```

```
    # Pour 3 -> 4
```

```
    pour = min(b, 4 - a) states.add((a + pour, b - pour))
```

```
    stack.extend(states) water_jug()
```

Output

0 0

0 3

4 3

4 0

1 3

1 0

0 1

4 1

2 3

Experiment: 3

Aim: Write a python program to remove punctuations from the given string?

Theory

Removing punctuation is an important step in preprocessing text.
Useful for NLP tasks like tokenization, classification, and sentiment analysis.
Can be done using:
Python string.punctuation
List comprehension
Regular expressions (regex)
Cleaned text improves accuracy of NLP models.

Complexity

Time Complexity: $O(n)$

Space Complexity: $O(n)$

code

```
punctuations = "!()-[]{};:,\\<>./?@#$$%^&*~_"  
text = input("Enter a string: ")  
result = ""
```

```
for char in text:
```

```
if char not in punctuations: result += char
```

```
print(result)
```

Output

```
Enter a string: HELLO#$%^  
HELLO
```

Experiment: 4

Aim: Write a python program to sort the sentence in alphabetical order?

Theory

Sorting words alphabetically means arranging them in *dictionary order*.

Steps:

- 1 Convert sentence into list of words.
- . Apply sorting using Python's sorted() or .sort().
- 2 Join words back into a sentence.
- . Sorting helps in indexing, text normalization, and keyword comparison.
- 3

Complexity

Time Complexity: $O(n \log n)$

Space Complexity: $O(n)$

```
sentence = "hello this is a python program"
```

```
words = sentence.split()
```

```
words.sort()
```

```
sorted_sentence = " ".join(words)
```

```
print(sorted_sentence)
```

Output

a hello is program python this

Experiment: 5

Aim: Write a program to implement Hangman game using python.

Theory

Hangman is a word-guessing game based on limited attempts.

User guesses letters; correct guesses reveal parts of the word.

Wrong guesses reduce remaining lives.

Requires:

- Random word selection
 - Loop until user wins or loses
 - String update for correct guesses
- Demonstrates *loops*, *conditionals*, *string handling*, and *lists*.

Complexity

Time Complexity: $O(n * g)$

(n = word length, g = number of guesses)

Space Complexity: $O(n)$

```
import random

words = ["python", "hangman", "computer", "science"] word
= random.choice(words)
guessed = ["_"] * len(word)
attempts = 6

while attempts > 0:
    print(" ".join(guessed))
    guess = input("Guess a letter: ")

    if guess in word:
        for i in range(len(word)): if word[i] ==
            guess:
                guessed[i] = guess
            else:
                attempts -= 1
        print("Wrong! Attempts left:", attempts)

    if "_" not in guessed:
        print("You win! Word was:", word) break
    else:
        print("You lost! Word was:", word)
```

Output

```
-----  
Guess a letter: p  
p _ _ _ _  
Guess a letter: y  
p y _ _ _  
Guess a letter: t  
p y t _ _ _  
Guess a letter: h  
p y t h _ _  
Guess a letter: o  
p y t h o _  
Guess a letter: n  
You win! Word was: python
```

Output

```
-----  
Guess a letter: a  
Wrong! Attempts left: 5  
-----  
Guess a letter: w  
Wrong! Attempts left: 4  
-----  
Guess a letter: t  
_ _ t _ _ _  
Guess a letter: i  
Wrong! Attempts left: 3  
_ _ t _ _ _  
Guess a letter: l  
Wrong! Attempts left: 2  
_ _ t _ _ _  
Guess a letter: k  
Wrong! Attempts left: 1  
_ _ t _ _ _  
Guess a letter: b  
Wrong! Attempts left: 0  
You lost! Word was: python
```

Experiment: 6

Aim: Write a program to implement Tic-Tac-Toe game using python.

Theory

A two-player game on a 3×3 grid.

Players take turns placing markers (X/O).

Game checks rows, columns, and diagonals for a winner.

Uses:

2D list (matrix)

- Input validation
- Loop until win or draw
- Example of implementing game logic using Python.

Complexity

Time Complexity:

~~O(1)~~ (Grid is always 3×3)

Space Complexity: O(1)

Code:

```
board=[" "] * 9
```

```
def print_board():
```

```
    print(board[0], "|", board[1], "|", board[2])
    print("--+---+--")
    print(board[3], "|", board[4], "|", board[5])
    print("--+---+--")
    print(board[6], "|", board[7], "|", board[8])
```

```
def check_win():
```

```
    win_pos = [
        (0,1,2), (3,4,5), (6,7,8),
        (0,3,6), (1,4,7), (2,5,8),
        (0,4,8), (2,4,6)
    ]
    for a,b, c in win_pos:
        if board[a] == board[b] == board[c] != " ":
            return True
    return False
```

```
player="X"
```

```
for _ in range(9):
```

```
    print_board()
```

```
    pos=int(input(f"Player {player}, enter position (0-8): "))
```

```

# if position is empty, place the symbol
if board[pos] == " ":
    board[pos] = player
else:
    print("Position already taken! Try again.")
    continue

# check for win
if check_win():
    print_board()
    print(f"Player {player} wins!")
    break

# switch player
player = "O" if player == "X" else "X"
else:
    print_board()
    print("It's a draw!")

```

Output

```

  |  |
--+--+
  |  |
--+--+
  |  |
Player X, enter position (0-8): 2
  |  | X
--+--+
  |  |
--+--+
  |  |
Player O, enter position (0-8):

```

Experiment: 7

Aim: Write a python program to remove stopwords for a given passage from a text file using NLTK?

Theory

Stop words are common words like *the, is, and, are*.

Removing them reduces noise in text processing.

NLTK provides a stop word corpus for many languages.

Helps in tasks like:

- Text classification
- Information retrieval
- Search engines

Improves performance by reducing dimensionality.

Complexity

Time Complexity: $O(n)$

Space Complexity: $O(n)$

```
[2] import nltk
    nltk.download('punkt')
    nltk.download('punkt_tab')
    nltk.download('stopwords')

... [nltk_data] Downloading package punkt to /root/nltk_data...
    [nltk_data] Unzipping tokenizers/punkt.zip.
    [nltk_data] Downloading package punkt_tab to /root/nltk_data...
    [nltk_data] Unzipping tokenizers/punkt_tab.zip.
    [nltk_data] Downloading package stopwords to /root/nltk_data...
    [nltk_data] Unzipping corpora/stopwords.zip.
    True

[7] from nltk.corpus import stopwords
    from nltk.tokenize import word_tokenize

    text = "This is a simple example demonstrating stop word removal."
    words = word_tokenize(text)

    filtered = [w for w in words if w.lower() not in stopwords.words('english')]

    print(filtered)

... ['simple', 'example', 'demonstrating', 'stop', 'word', 'removal', '.']
```

Experiment: 8

Aim: Write a python program to implement stemming for a given sentence using NLTK?

Theory

Stemming reduces words to their base/root form.

Example: *playing* → *play*, *studies* → *studi*.

NLTK provides PorterStemmer, SnowballStemmer, etc.

Mainly used in:

Search engines

- Text mining
- Information retrieval

Produces fast but sometimes rough results.

Complexity

Time Complexity: $O(n)$

Space Complexity: $O(n)$

```
[2] import nltk
    nltk.download('punkt')
    nltk.download('punkt_tab')
    nltk.download('stopwords')

... [nltk_data] Downloading package punkt to /root/nltk_data...
    [nltk_data] Unzipping tokenizers/punkt.zip.
    [nltk_data] Downloading package punkt_tab to /root/nltk_data...
    [nltk_data] Unzipping tokenizers/punkt_tab.zip.
    [nltk_data] Downloading package stopwords to /root/nltk_data...
    [nltk_data] Unzipping corpora/stopwords.zip.
    True

[6] from nltk.stem import PorterStemmer
    from nltk.tokenize import word_tokenize

    ps = PorterStemmer()
    sentence = "running runs runner easily fairly played playing"
    words = word_tokenize(sentence)

    stemmed = [ps.stem(word) for word in words]
    print(stemmed)

... ['run', 'run', 'runner', 'easili', 'fairli', 'play', 'play']
```

Experiment: 9

Aim: Write a python program to POS (Part of Speech) tagging for the give sentence using NLTK?

Theory

POS (Parts of Speech) tagging labels each word as noun, verb, adjective, etc. NLTK uses pretrained models for tagging.

Required for:

- Parsing
- Named Entity Recognition
- Machine translation
- Helps understand the grammatical structure of sentences.

Complexity

Time Complexity: $O(n)$

Space Complexity: $O(n)$

```
[9] import nltk
    nltk.download('punkt')
    nltk.download('punkt_tab')
    nltk.download('averaged_perceptron_tagger_eng')

... [nltk_data] Downloading package punkt to /root/nltk_data...
    [nltk_data] Package punkt is already up-to-date!
    [nltk_data] Downloading package punkt_tab to /root/nltk_data...
    [nltk_data] Package punkt_tab is already up-to-date!
    [nltk_data] Downloading package averaged_perceptron_tagger_eng to
    [nltk_data] /root/nltk_data...
    [nltk_data] Unzipping taggers/averaged_perceptron_tagger_eng.zip.
    True

[10] from nltk import pos_tag, word_tokenize

    text = "This is a test sentence."
    words = word_tokenize(text)

    pos = pos_tag(words)
    print(pos)

... [('This', 'DT'), ('is', 'VBZ'), ('a', 'DT'), ('test', 'NN'), ('sentence', 'NN'), ('.', '.')]

```

Experiment: 10

Aim: Write a python program to implement Lemmatization using NLTK?

Theory

Lemmatization reduces words to a valid dictionary root (lemma).

Example: *better* → *good*, *studies* → *study*.

More accurate than stemming but slower.

Uses WordNetLemmatizer from NLTK.

Used in advanced NLP tasks:

- Chatbots
- Machine learning preprocessing
- Semantic analysis

Complexity

Time Complexity: $O(n)$

Space Complexity: $O(n)$

```
[12]
✓ 0s
import nltk
nltk.download('punkt')
nltk.download('punkt_tab')
nltk.download('wordnet')
nltk.download('omw-1.4')

... [nltk_data] Downloading package punkt to /root/nltk_data...
[nltk_data] Package punkt is already up-to-date!
[nltk_data] Downloading package punkt_tab to /root/nltk_data...
[nltk_data] Package punkt_tab is already up-to-date!
[nltk_data] Downloading package wordnet to /root/nltk_data...
[nltk_data] Downloading package omw-1.4 to /root/nltk_data...
True

[13]
✓ 3s
from nltk.stem import WordNetLemmatizer
from nltk.tokenize import word_tokenize

lemmatizer = WordNetLemmatizer()

text = "running playing better"
words = word_tokenize(text)

print([lemmatizer.lemmatize(w, pos="v") for w in words])

... ['run', 'play', 'better']
```

Experiment: 11

Aim: Write a python program for Text Classification for the give sentence using NLTK?

Theory

Process of assigning labels (positive/negative/spam/category) to text.

Requires:

- Tokenization
- Stop word removal
- Feature extraction (Bag of Words / TF-IDF)
- Classifier training (Naive Bayes commonly used in NLTK)

NLTK provides:

- Naive Bayes classifier
- Decision Tree classifier

Used in:

- Spam detection
- Sentiment analysis
- Document categorization

Complexity

Depends on classifier:

Training Time: $O(n * f)$

(n = number of samples, f = number of features)

Prediction Time: $O(f)$

Space Complexity: $O(n + f)$

```
[17] 0s # Simple Text Classification without NLTK datasets

positive_words = ["good", "happy", "excellent", "great", "love", "awesome"]
negative_words = ["bad", "sad", "terrible", "hate", "poor", "worst"]

def classify(text):
    text = text.lower()
    pos_count = sum(word in text for word in positive_words)
    neg_count = sum(word in text for word in negative_words)

    if pos_count > neg_count:
        return "Positive 😊"
    elif neg_count > pos_count:
        return "Negative 😞"
    else:
        return "Neutral 😐"

# Test
sentence = "The movie was excellent and I love the acting!"
print("Sentence:", sentence)
print("Sentiment:", classify(sentence))

*** Sentence: The movie was excellent and I love the acting!
    Sentiment: Positive 😊
```